Data Flow Diagram

for

Website Vulnerability Scanner

Prepared by

Deepthi K (241IT021)
Jasmine (241IT051)
Sacheth Koushal (241IT058)

Department of Information Technology
National Institute of Technology Karnataka, Surathkal

September 2026

Design artifact derived from the canonical WVS SRS v1.2. It does not override the SRS.

Contents

1 Data Flow Diagram (DFD) Model

1.1 Level 0 DFD

1.1.1 Context Diagram

Figure 1 shows the Level 0 Context Diagram for the Website Vulnerability Scanner system. It defines the main system boundary for Process 0 and models all inbound/outbound interaction channels with main external entities such as the WVS User, System Administrator, Target Website, Public DNS Resolvers, SMTP Relay, and OSV/EPSS Advisory APIs.

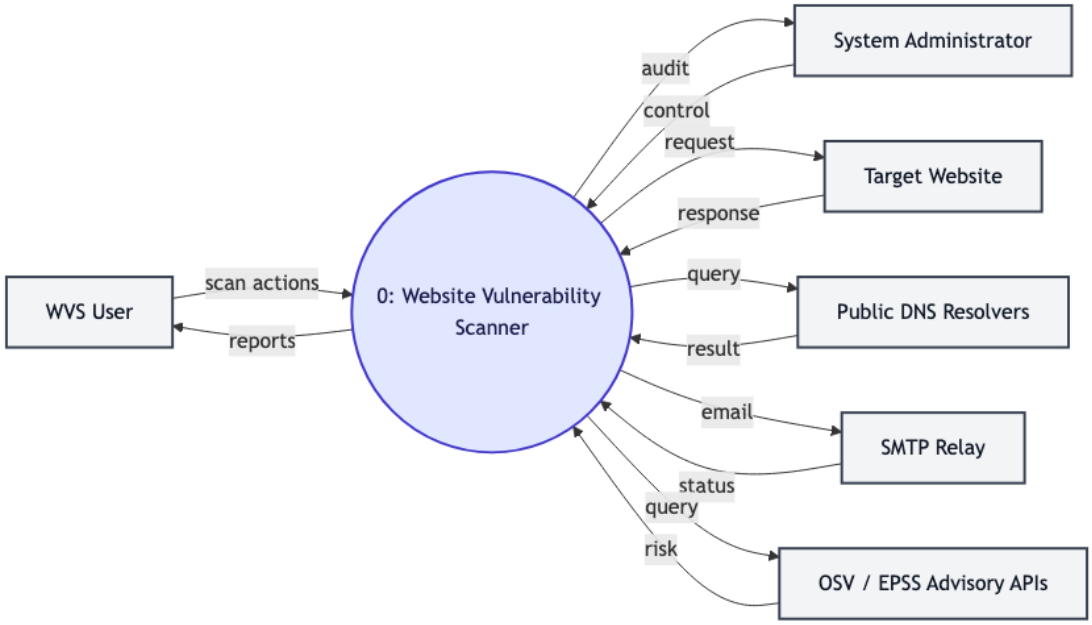

Figure 1: Level 0 Context Diagram – the entire system as a single process, showing only external entities

1.2 Level 1 DFD

1.2.1 Top-Level Functional Decomposition

Figure 2 shows the Level 1 DFD. Process 0 is decomposed into 8 primary functional modules (0.1 through 0.8). It displays the main scanning pipeline from the initial user request through scope validation, discovery crawling, vulnerability detection, findings management, and report generation, as well as centralized Scope Guard enforcement (0.8) and third-party integrations.

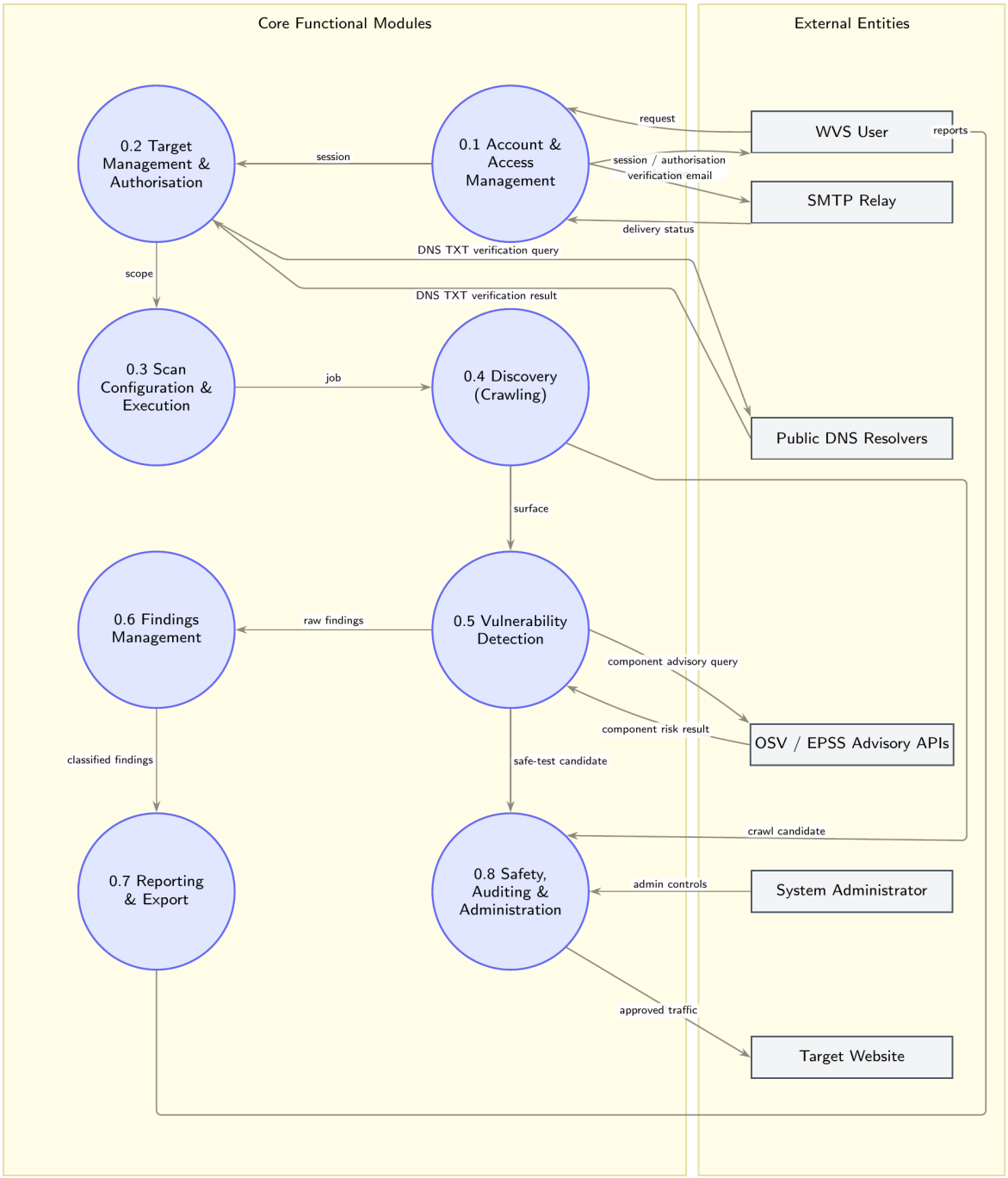

Figure 2: Level 1 DFD – process 0 decomposed into 8 functional groups

1.3 Level 2 DFDs

1.3.1 0.1 User Authentication & Access Control

Process 0.1 is decomposed into credential validation, session token issuance, and login audit logging. Process 0.1.1 validates the user credentials by sending a verification request to Process 0.1.2 and saves the account and failed attempts in Data Store D1, Process 0.1.2 then issues session tokens and verifies email deliverability via SMTP Relay, and Process 0.1.3 records authentication audits.

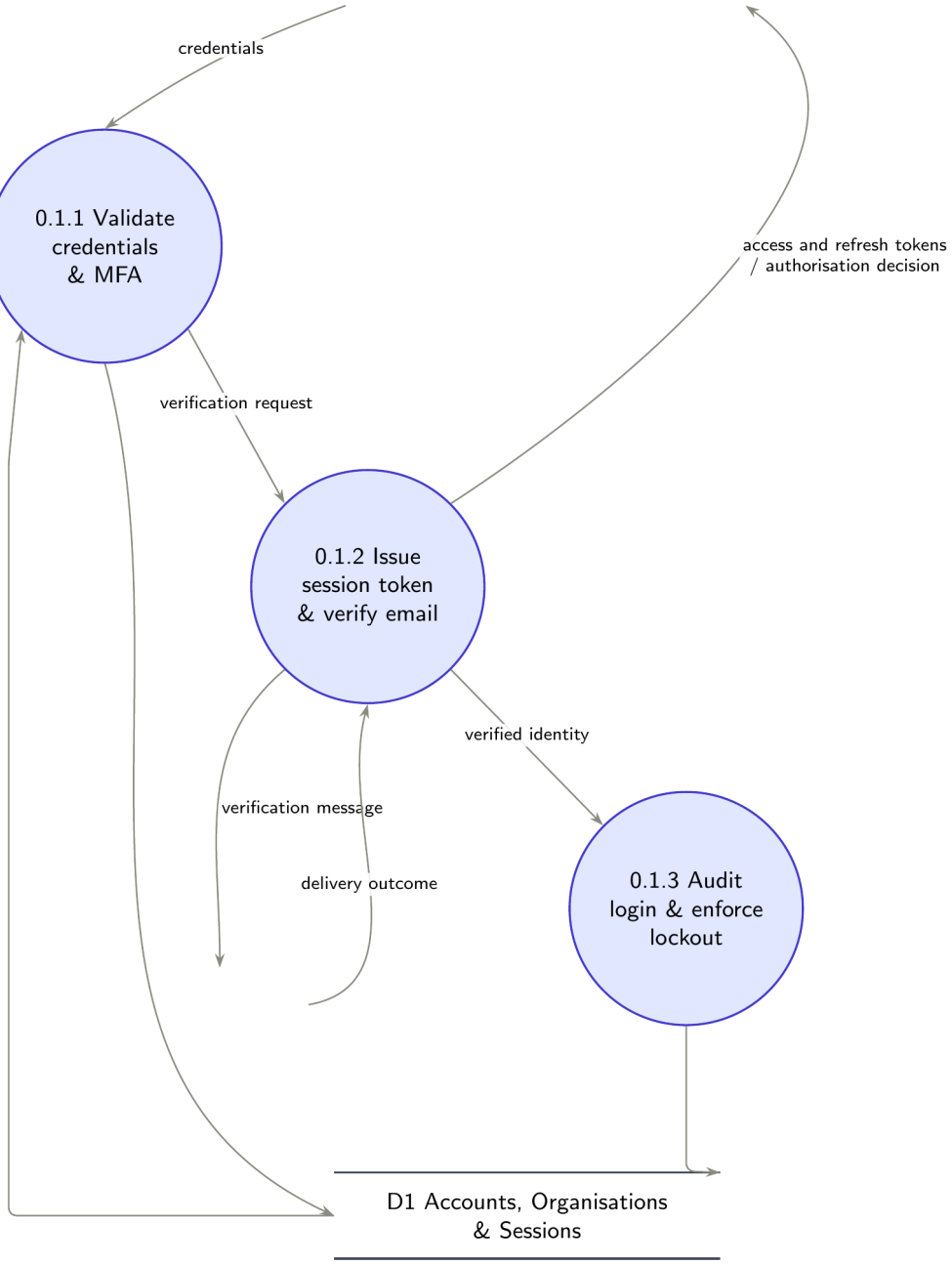

Figure 3: Level 2 DFD for 0.1 User Authentication & Access Control

1.3.2 0.2 Target Management & Authorisation

Process 0.2 is decomposed into target registration, verification token generation, and DNS/well-known proof validation. Process 0.2.1 registers target records in D2, Process 0.2.2 issues verification tokens for TXT/well known path validation and Process 0.2.3 validates these tokens against Public DNS and the Target Website.

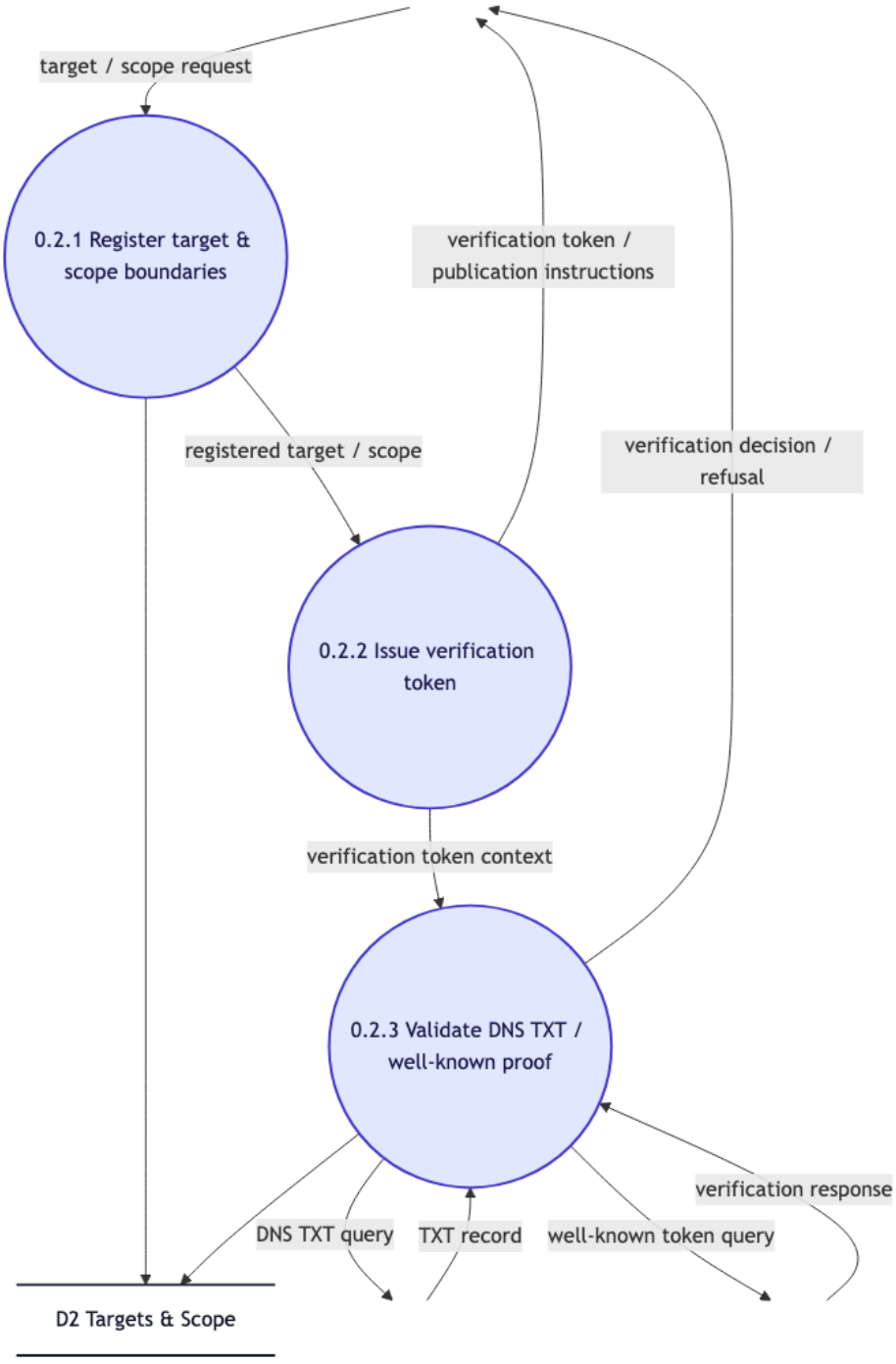

Figure 4: Level 2 DFD for 0.2 Target Management & Authorisation

1.3.3 0.3 Scan Configuration & Execution

Process 0.3 is decomposed into scan profile validation, job scheduling, and lifecycle execution. Process 0.3.1 verifies user scope and quotas against D2 and Process 0.3.2 queues scan jobs in D3, and Process 0.3.3 executes the scan lifecycle while writing reproducibility records to D4.

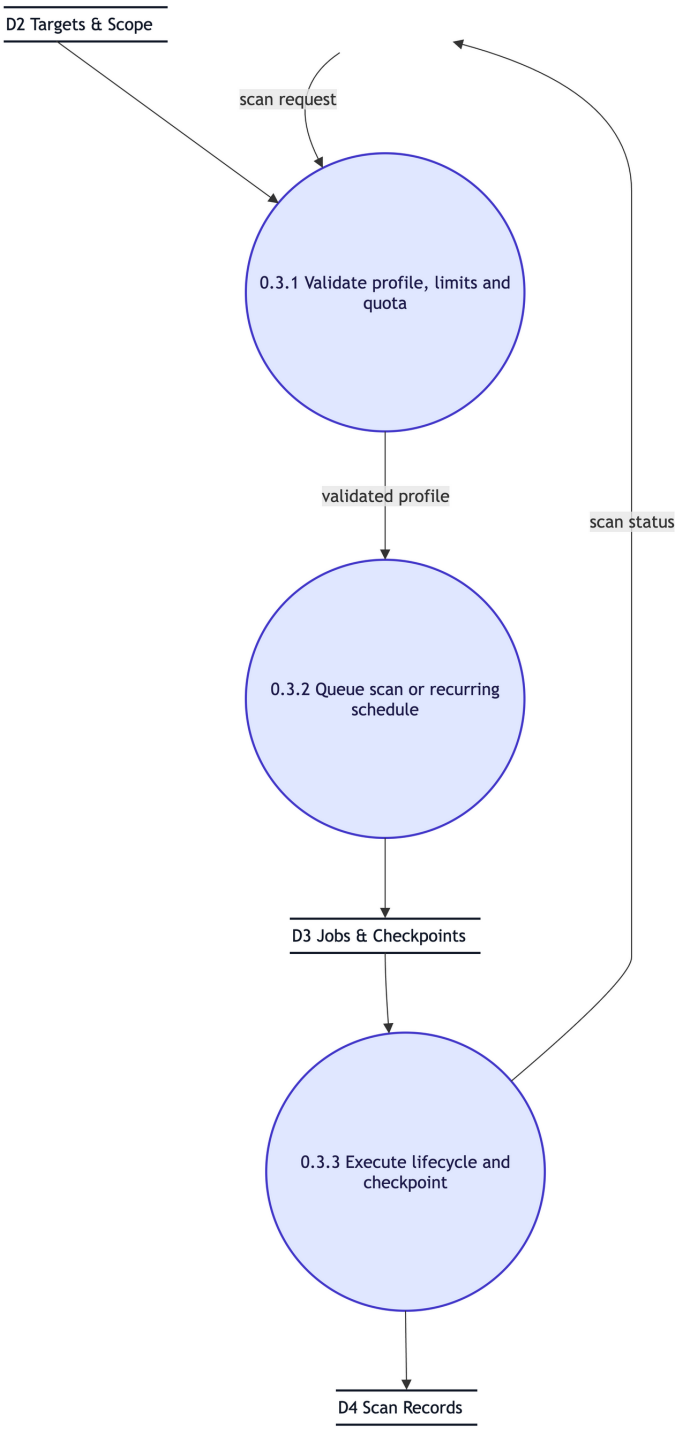

Figure 5: Level 2 DFD for 0.3 Scan Configuration & Execution

1.3.4 0.4 Discovery (Crawling)

Process 0.4 is decomposed into scope loading, link/form discovery, and JS rendering deduplication. Process 0.4.1 loads ceilings from D2, Process 0.4.2 sends crawl candidates to Process 0.8 Scope Guard, and Process 0.4.3 records page inventory in D4 upon receiving HTTP responses from the

Target Website.

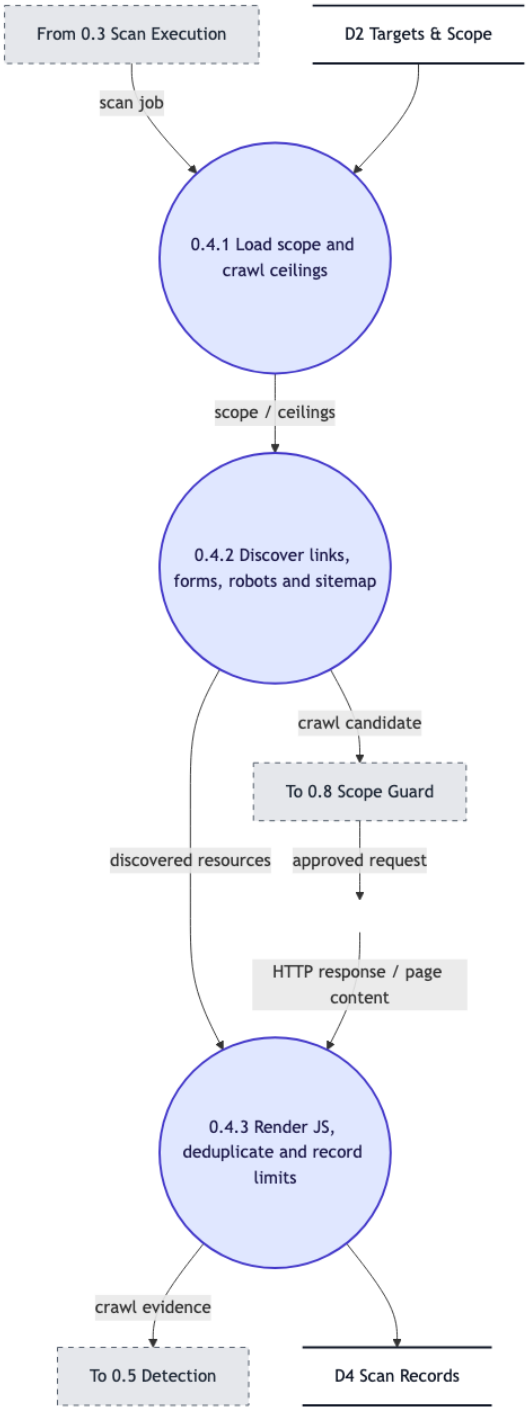

Figure 6: Level 2 DFD for 0.4 Discovery (Crawling)

1.3.5 0.5 Vulnerability Detection

Process 0.5 is decomposed into passive header checking, safe active test preparation, and advisory version enrichment. Process 0.5.1 runs passive analysis, Process 0.5.2 routes active test payloads through Process 0.8 Scope Guard, and Process 0.5.3 queries OSV/EPSS APIs and stores evidence in D5.

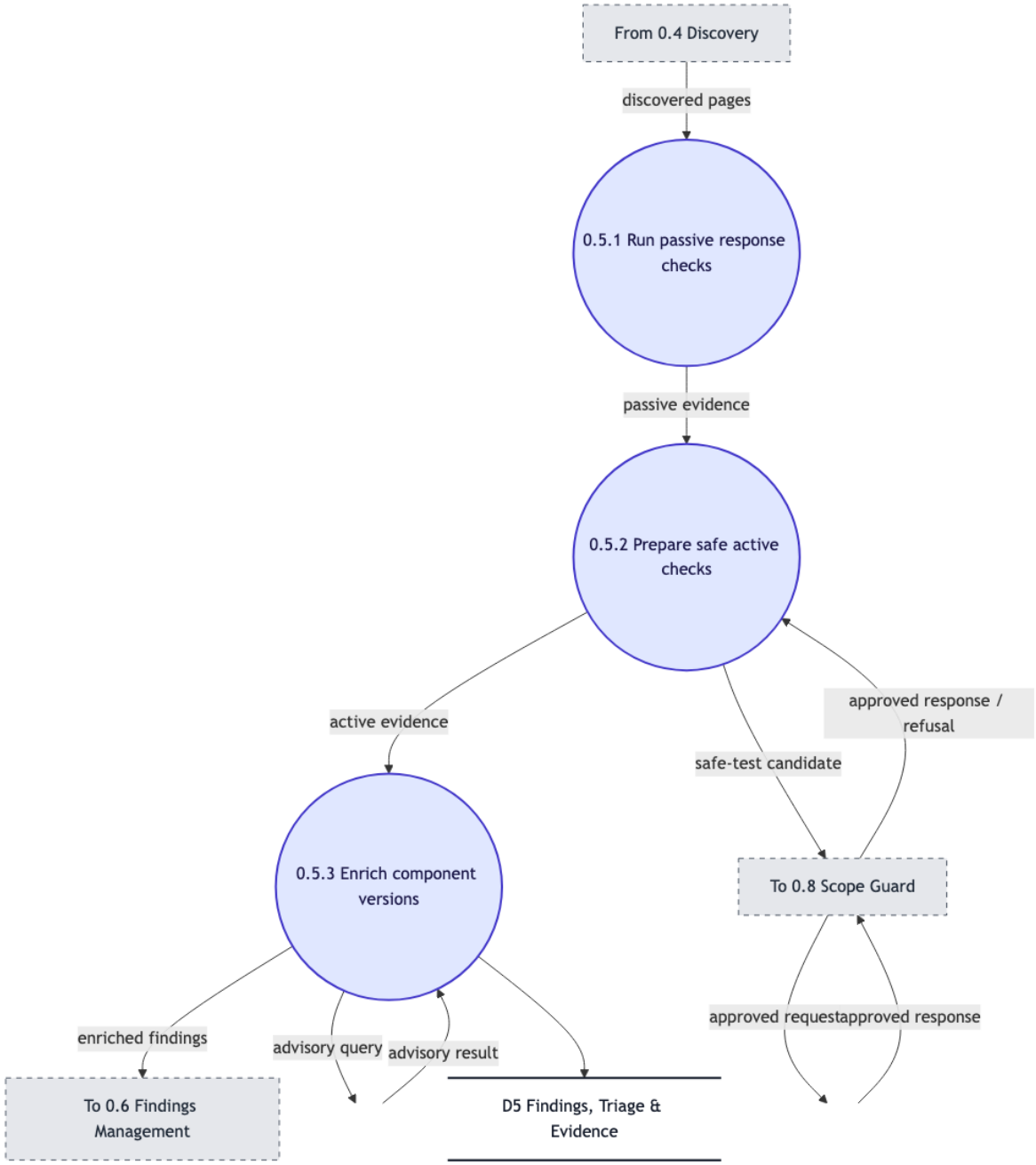

Figure 7: Level 2 DFD for 0.5 Vulnerability Detection

1.3.6 0.6 Findings Management

Process 0.6 is decomposed into normalisation/redaction, prior scan comparison, evidence retention and triage filtering. Process 0.6.1 ingests raw findings, normalises, redacts and deduplicates them. Process 0.6.2 compares prior scans stored in D5 and retains evidence, and Process 0.6.3 applies user triage, filters and sorts them and also implements severity classification.

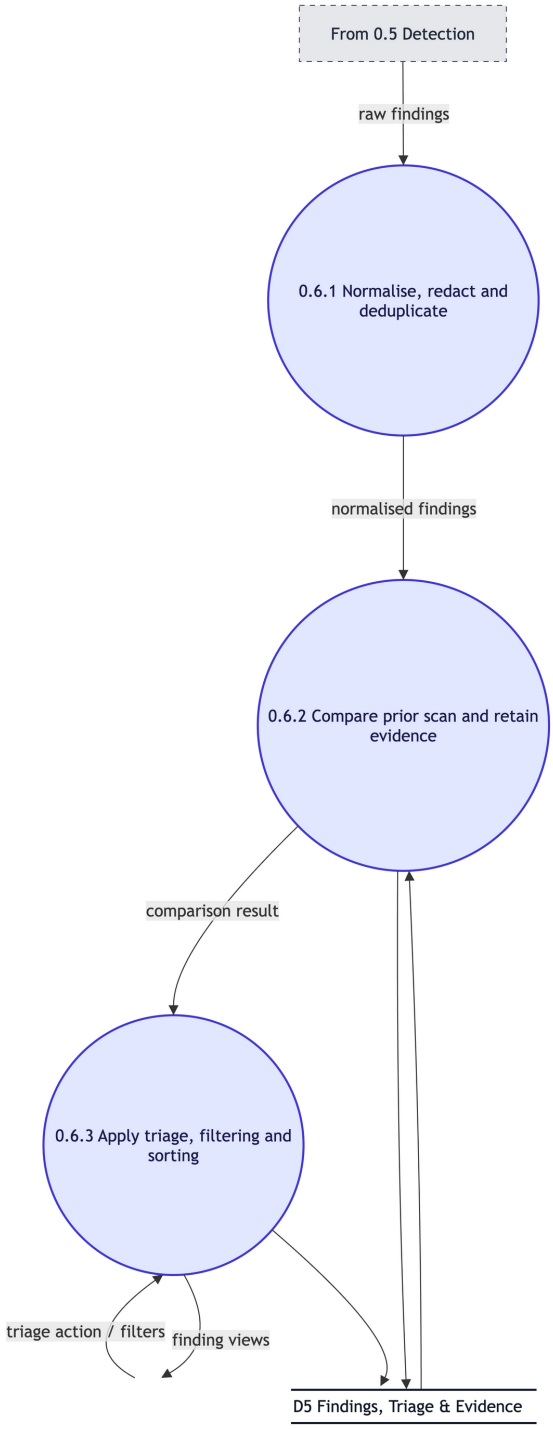

Figure 8: Level 2 DFD for 0.6 Findings Management

1.3.7 0.7 Reporting & Export

Process 0.7 is decomposed into completed scan selection, report template assembly, and export dispatch. Process 0.7.1 retrieves scan metadata from D4, Process 0.7.2 assembles the template, filters and limitations and the findings, triage and evidence from D5, and Process 0.7.3 generates PDF, HTML, JSON, CSV, and SARIF exports and triggers SMTP notifications.

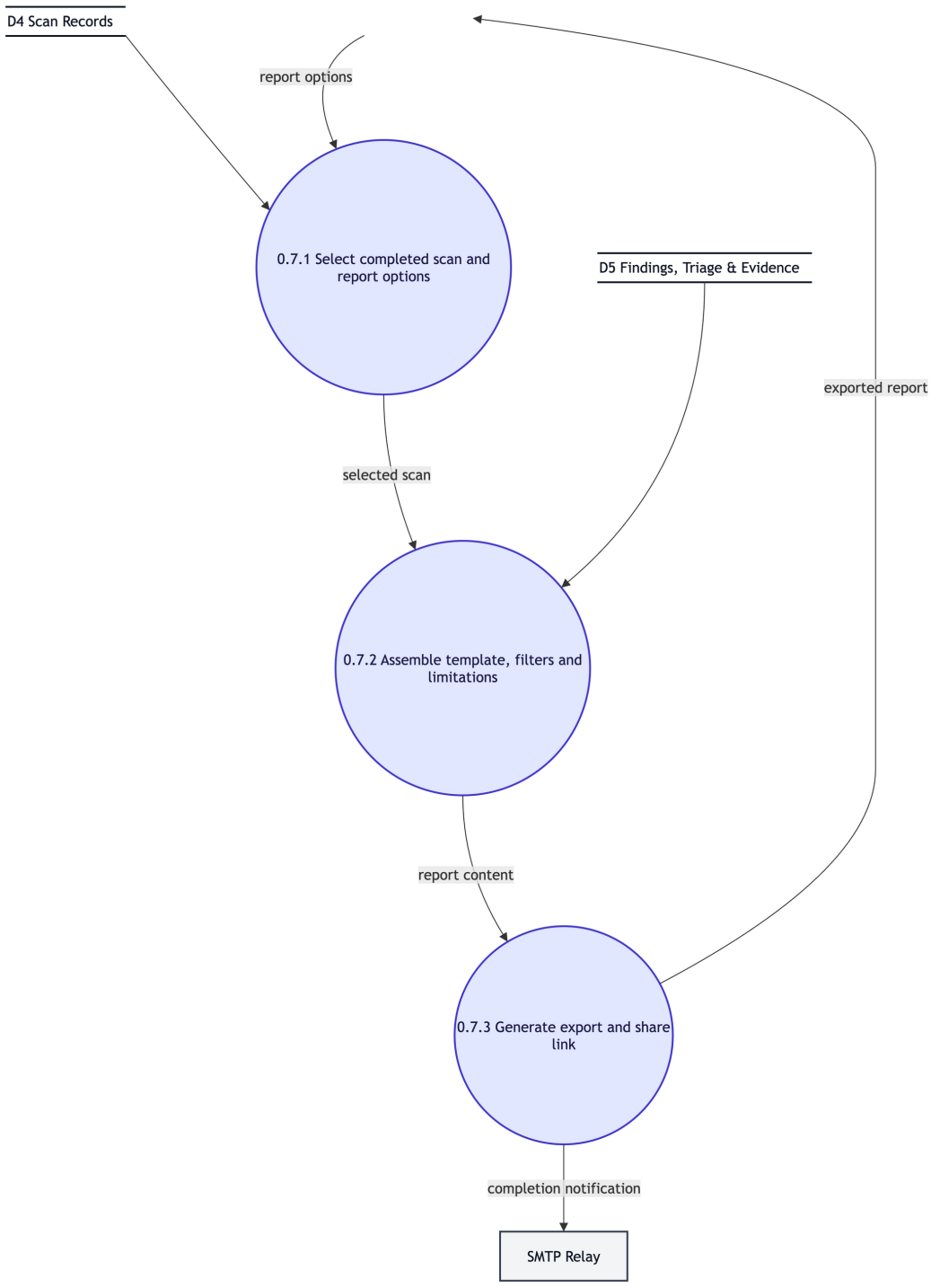

Figure 9: Level 2 DFD for 0.7 Reporting & Export

1.3.8 0.8 Safety, Auditing & Administration

Process 0.8 is decomposed into scope/blocklist verification, IP re-resolution & rate limiting, and request dispatch. Process 0.8.1 checks requests against D2 policies. Process 0.8.2 performs fresh

DNS checks to prevent SSRF, and Process 0.8.3 logs dispatches to D6 Append-only Audit Ledger.

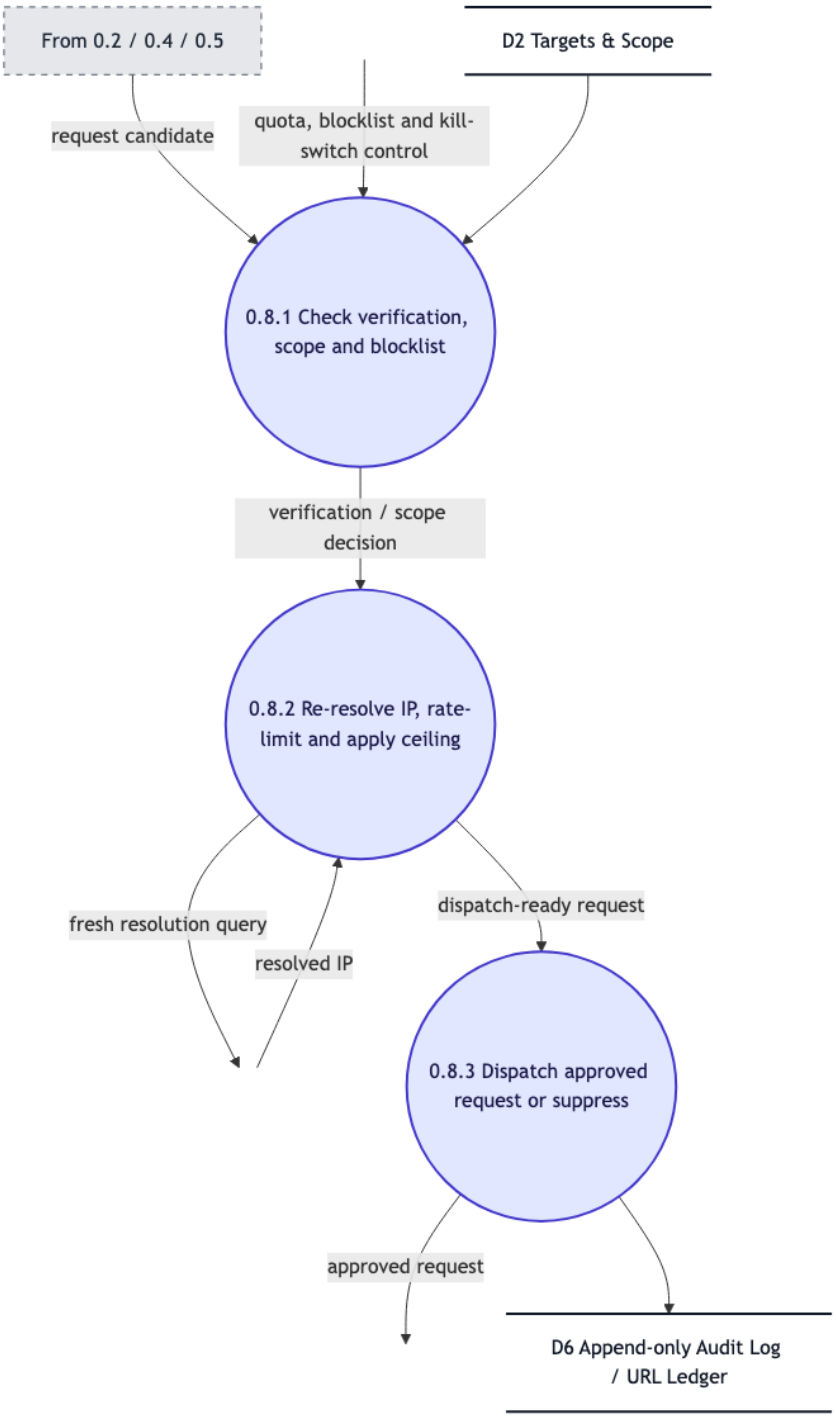

Figure 10: Level 2 DFD for 0.8 Safety, Auditing & Administration

2 Core Modules

This section explains the main functional modules of the Website Vulnerability Scanner. Each module has been assigned to a designer for detailed implementation, and the functional requirements (from the SRS) realised by each module are listed.

2.1 0.1 User Authentication & Access Control

This module manages user login, account registration, organisation onboarding, active sessions, and multi-factor authentication. It also helps protect user accounts by tracking login attempts and restricting access after repeated failures.

- **Designer**: Deepthi K
- **Functions**: Register users, verify login credentials and MFA, generate session tokens (JWT), record login activity, and apply account lockout when required.
- **Requirements Covered**: F.1

2.2 0.2 Target Management & Authorisation

This module allows users to register the websites or domains they want to scan and define the permitted scanning scope. It also verifies that the user has permission to scan the target using DNS TXT records or well-known verification files.

- **Designer**: Deepthi K
- **Functions**: Register targets and scan scope, generate verification tokens, validate DNS TXT or well-known proofs, and store scope rules.
- **Requirements Covered**: F.2

2.3 0.3 Scan Configuration & Execution

This module manages the configuration and execution of vulnerability scans. It checks whether the selected scan settings are valid, verifies user limits, schedules scan jobs, and keeps track of each stage of the scan.

- **Designer**: Deepthi K
- **Functions**: Validate scan profiles and quotas, queue or schedule scan jobs, manage scan execution and checkpoints, and store information required to reproduce a scan.
- **Requirements Covered**: F.3

2.4 0.4 Discovery (Crawling)

This module explores the authorised website to identify possible attack surfaces such as links, forms, API endpoints, and dynamically generated JavaScript content. The crawler follows the limits and scope defined for the scan.

- **Designer**: Jasmine
- **Functions:**
  Load scope and crawling limits, discover links and sitemap entries, render JavaScript content, remove duplicate results, and send candidates to the Scope Guard for validation.
- **Requirements Covered**: F.4

2.5 0.5 Vulnerability Detection

This module checks the discovered website components for security weaknesses. It performs passive checks on responses and headers, carries out safe active tests only on authorised targets, and uses external security databases to identify known vulnerabilities.

- **Designer**: Jasmine
- **Functions**: Perform passive response checks, prepare safe active vulnerability tests, and check component versions using OSV and EPSS APIs.
- **Requirements Covered**: F.5

2.6 0.6 Findings Management

This module processes the raw results produced during a scan and converts them into clear findings. It removes or hides sensitive information, compares findings with previous scans, and organises the results based on their severity and status.

- **Designer**: Jasmine
- **Functions**: Normalise and redact findings, compare results with previous scan evidence, apply triage rules, filter findings, and sort them by severity.
- **Requirements Covered**: F.6

2.7 0.7 Reporting & Export

This module converts completed scan results into readable reports for both technical and non-technical users. It also supports exporting scan information and notifying users when a scan is complete.

- **Designer**: Sacheth Koushal
- **Functions**: Select completed scans, prepare report templates and supporting evidence, generate PDF or CSV exports and share links, and send completion notifications through SMTP.
- **Requirements Covered**: F.7

2.8 0.8 Safety, Auditing & Administration

This module ensures that scans remain within the approved scope and follow the required safety rules. It checks target verification, blocks restricted destinations, re-checks IP addresses to reduce SSRF and TOCTOU risks, limits request rates, and records scanning activity for auditing.

- **Designer**: Sacheth Koushal
- **Functions**: Verify target authorisation and scope blocklists, re-resolve IP addresses, apply rate limits, send approved requests, and maintain an append-only audit log.
- **Requirements Covered**: F.8

3 Data Dictionary

A single, consolidated data dictionary is given below, covering the data stores and principal data flows appearing across all DFDs in Section 1 (Level 0 through Level 2). Composition operators used: + (composition), [,] (selection), ( ) (optional data), { } (iteration), = (equivalence), and /* */ (comment).

3.1 Data Stores

Data Store	Definition
D1 Accounts, Organisations & Sessions	accounts-store = {username + email + password-hash + MFA-status + organisation-membership + session-token + failed-login-count}*
D2 Targets & Scope	targets-store = {target-domain + verified-IP-ranges + path-inclusion-rules + path-exclusion-rules + ownership-verification-token + scan-ceiling}*
D3 Jobs & Checkpoints	jobs-store = {scan-job + worker-assignment + execution-state + pause-checkpoint + cron-trigger}*
D4 Scan Records	scan-records = {scan-metadata + crawled-page-inventory + HTTP-request-headers + HTTP-response-headers + crawl-limits}*
D5 Findings, Triage & Evidence	findings-store = {vulnerability-record + CVSS-score + request-payload + response-payload + triage-state + comparison-metadata}*
D6 Append-only Audit Log / URL Ledger	audit-log = {outbound-request-record + timestamp + scope-guard-evaluation + kill-switch-event}* /* immutable, append-only */

3.2 Data Flows

Data Flow	Definition / Composition
`request`	request = [login-credentials, session-refresh-request]
`session`	session = session-token + organisation-permission-context
`scope`	scope = verified-target + path-boundaries + scan-authorisation
`job`	job = validated-scan-profile + schedule
`surface`	surface /* attack surface */ = {discovered-URL + forms + headers + JS-assets}*
`raw findings`	raw-findings = {vulnerability-candidate + request-payload + response-payload}*
`classified findings`	classified-findings = {normalised-finding + severity + triage-state}*
`reports`	reports = [pdf-report, html-report, json-export, csv-export, sarif-report] + (share-link)
`crawl candidate`	crawl-candidate = HTTP-request /* pending Scope Guard check */
`safe-test candidate`	safe-test-candidate = active-test-payload /* pending Scope Guard check */
`approved traffic`	approved-traffic = validated-HTTP-request /* dispatched within allowed rate limits */

3.3 Primitive Data Item Types

Primitive	Type
username, email	string
password-hash	string /* stored only as a salted hash, never in plaintext */
session-token	string /* JWT */
timestamp	datetime
severity	[info, low, medium, high, critical]
triage-state	[open, confirmed, false-positive, accepted-risk, resolved]
scan-ceiling	integer /* maximum pages/requests per scan */
HTTP-request, HTTP-response	{byte}*

4 Structured Design: Structure Chart

4.1 Level 1 Structure Chart

4.1.1 Module Decomposition

The structure chart is derived from the Level 1 and Level 2 DFDs using the structured design approach of Rajib Mall. The user-facing request routing is factored by transaction analysis into one action module per transaction path, and the scan pipeline ($0.3 \rightarrow 0.4 \rightarrow 0.5 \rightarrow 0.6$) is factored by transform analysis into an afferent branch, a transform centre, and an efferent branch. The Scope Guard module (0.8) appears as a shared library module (double-edged box), invoked by both the attack-surface discovery and the vulnerability detection modules to validate crawl and test candidates, with the control couple carrying back the approved response / refusal decision. Every data couple is labelled with the exact flow name used in the DFDs of Section 1 and defined in the data dictionary of Section 3, and points in the direction the data travels. Figure 11 illustrates the resulting module hierarchy.

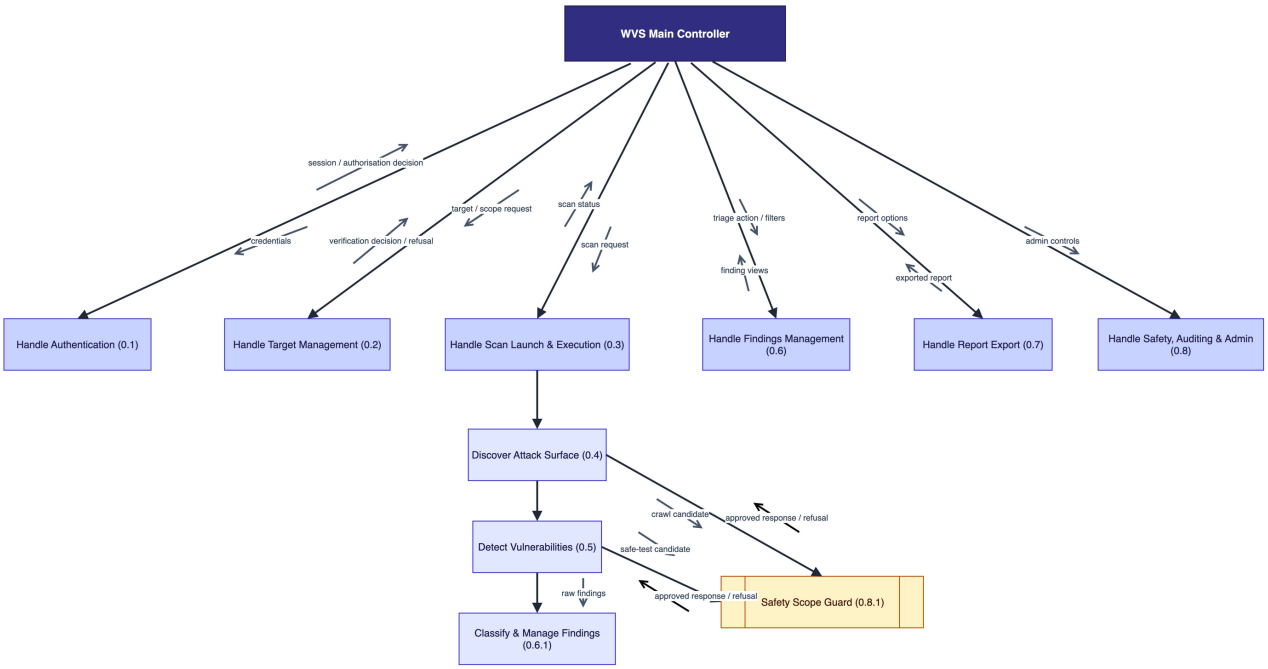

Figure 11: Level 1 Structure Chart of the System